



Simulation-based deep reinforcement learning for process time optimization in semiconductor cluster tool and an empirical study[☆]

Tzu-Yen Hong^a , Chia-Yu Hsu^{b,*} , Chia-Ling Chen^a

^a Department of Industrial Engineering and Management, National Taipei University of Technology, Taiwan

^b Department of Industrial Engineering & Engineering Management, National Tsing Hua University, Taiwan

ARTICLE INFO

Keywords:

Controllable processing time
Deep reinforcement learning
Discrete event simulation
Cluster tool
Semiconductor manufacturing

ABSTRACT

Semiconductor manufacturing is increasingly challenged by high product mix and significant operational variability. While prior research has mainly focused on production scheduling to minimize machine downtime, optimize resource allocation, and enhance maintenance efficiency, the studies on the dynamic adjustment of controllable process times in cluster tools such as physical vapor deposition (PVD) systems remains limited. This study proposes a simulation-integrated deep reinforcement learning framework that adjusts processing times to regulate wafer flow and reduce downstream waiting. A discrete-event simulation model of a PVD cluster tool is developed to accurately represent equipment interdependencies, recipe routings, and capacity constraints. A deep deterministic policy gradient (DDPG) agent is trained within this virtual environment to generate process time policies that minimize wafer waiting time and cycle time while aligning with target makespan objectives. Empirical evaluations using real production data and adapted benchmark scenarios demonstrate that the proposed approach yields substantial performance gains without narrowing the feasible solution space, in which achieving reductions of 31.54% in wafer waiting time and 13.46% in cycle time. The results show the potential of adaptive, learning-based decision support systems in enhancing operational efficiency in advanced semiconductor fabrication.

1. Introduction

The semiconductor industry is experiencing growing complexity driven by diverse customer demands, stringent quality requirements, shorter product life cycles, and intensified global competition. These factors significantly increase the difficulty of production planning. Indeed, wafer fabrication involves a series of complex, repetitive operations, including oxidation, deposition, metallization, lithography, etching, ion implantation, resist stripping, cleaning, inspection, and measurement (Chien & Chen, 2007; Chien, et al. 2020). The production system is characterized by high product variety, where multiple products with distinct specifications are processed concurrently. Although each product follows a specific routing and process sequence, they often share common equipment. This resource sharing requires synchronized scheduling and coordination among different product flows to ensure efficient utilization. Thus, effective control of process time, resource allocation, and workflow coordination becomes essential. These factors

influence operational efficiency, and cost reduction. However, capacity expansion through new fabs or equipment is capital-intensive and time-consuming, leading to long investment cycles and uncertainty (Chen & Chien, 2011). Therefore, enhancing production efficiency through existing resource optimization remains a key challenge (Hong & Chien, 2020).

In semiconductor manufacturing, machine processing times are generally modeled as deterministic and constant values in batch production environments (Li & Chang, 2022). Consequently, improvements in productivity usually rely on production scheduling adjustments. However, the stochastic characteristics and inter-machine dependencies result in a large and complex state space, encompassing variables such as cycle times, machine status, and wafer transfer. The limitation of traditional mathematical programming approaches can be traced to express these characteristics (Li & Chang, 2022). While previous studies have focused on downtime reduction, maintenance planning, or resource allocation, relatively few have explored the use of process time

[☆] This article is part of a special issue entitled: 'AI for Semiconductor manufacturing' published in Computers & Industrial Engineering.

* Corresponding author at: Department of Industrial Engineering & Engineering Management, National Tsing Hua University, 101 Section 2 Kuang Fu Road, Hsinchu 30013, Taiwan.

E-mail address: cyhsu@ie.nthu.edu.tw (C.-Y. Hsu).

<https://doi.org/10.1016/j.cie.2025.111698>

adjustment as a means to improve throughput in semiconductor manufacturing. However, some machine tools are often equipped with adjustable parameters that allow fine control over processing time. For instance, in PVD process, the processing time can often be adjusted in the recipe without altering other key process parameters such as power, chamber pressure, or gas flow rates. The control mechanism provides a simple means of fine-tuning the temporal dynamics of wafer processing, especially when film thickness tolerances allow minor adjustments in deposition duration without compromising product quality. These adjustments are commonly employed in scenarios where the product specification allows greater variation in film thickness, the production system aims to smooth out fluctuations in work-in-progress (WIP) accumulation or buffer queue lengths. The objective is not to achieve a target thickness but rather to maintain synchronization with upstream or downstream equipment. By slightly extending or shortening the deposition duration, the system can regulate wafer flow pacing and reduce machine idleness or downstream waiting times. For instance, when a downstream tool experiences a temporary delay or is reaching its capacity limit, increasing the sputtering time on the current tool provides a buffer to absorb such disturbances without the need to reschedule jobs.

This controllability arises from the continuous and deterministic relationship between input parameters and material transformation rates, enabling adaptive runtime decisions without violating product quality requirements. Several studies have explored the use of processing time as a decision variable to improve production performance. [Shabtay and Steiner \(2007\)](#) provided a comprehensive survey on scheduling problems with controllable processing times, highlighting various models and solution approaches. [Shabtay and Zofi \(2018\)](#) examined single-machine scheduling problems where job processing times are controllable, addressing scenarios with fixed machine unavailability intervals, in which adjusting processing times can lead to more efficient scheduling outcomes, particularly in environments with resource constraints. Indeed, the empirical implementation of controllable processing time is also widely considered in the scheduling application of CNC manufacturing ([Atan & Akturk, 2008](#); [Akturk & Ilhan, 2011](#)). While previous studies show the importance in practice. Most studies focus on single machine or small-scale problems ([Cheng et al. 2024](#)). Scheduling algorithms may often be limited by computational costs and delayed responsiveness to real-time disturbances for a practical manufacturing system. There have been a few studies that consider the above characteristics within the context of the semiconductor industry ([Hu et al. 2023](#); [Zhang et al. 2024](#)). Moreover, most existing studies focus on conventional scheduling objectives, such as minimizing makespan. Such objectives may conflict with the practical need to regulate wafer flow pacing and reduce machine idleness or downstream waiting times. In particular, minimizing makespan may also lead to imbalanced tool utilization, increased congestion at downstream stages, or extended idle periods for certain equipment.

Focusing on the practical needs, this study proposes a DRL-based approach for optimizing machine process times for PVD in wafer fabrication with realistic constraints. A discrete event simulation (DES) model is developed to represent production environments. By capturing the interdependencies among machines and evaluating system performance under varying process time configurations, the DRL model explores feasible processing time assignments for each job within a reasonable range of decomposition time, aiming to approximate the target total processing duration. This enables the system to regulate wafer flow more effectively and mitigate downstream waiting times, thereby improving synchronization across tools in a dynamic production environment. The main contributions of this work are as follows: (1) development of a simulation-based model to represent realistic semiconductor production processes with controllable process times; (2) application of DRL to learn optimal process configurations considering system-wide interdependencies; and (3) implementation of data-driven operational guidelines to support process engineers in decision-making.

The remainder of this paper is structured as follows. Section 2 reviews relevant literature. Section 3 presents the problem formulation and the proposed framework. Section 4 provides the experiments of the proposed approach. Section 5 concludes the paper and discusses future research directions.

2. Literature review

2.1. Scheduling with controllable processing time

The scheduling problem with controllable processing times has emerged as a significant research topic for flexible production systems. [Shabtay and Steiner \(2007\)](#) provided a comprehensive survey of scheduling problems with controllable processing times, establishing that processing times can be modeled as non-increasing functions of allocated resources and that integrating resource allocation decisions with scheduling yields significant improvements in system performance. Based on the foundation of the survey, [Shabtay and Zofi \(2018\)](#) addressed single-machine scheduling with controllable processing times and machine unavailability periods, proposing fully polynomial time approximation schemes to minimize makespan. [Shabtay \(2022\)](#) further incorporated resource-dependent processing times into models with fixed machine unavailability intervals, offering pseudo-polynomial algorithms and practical heuristics. Indeed, complementary studies have explored single-machine and parallel-machine environments. [Yang et al. \(2014\)](#) examined the assignment of multiple due windows under controllable processing times, demonstrating that optimized due window positions and processing compressions can jointly minimize earliness, tardiness, and compression costs. [Hsieh et al. \(2015\)](#) addressed unrelated parallel-machine scheduling with discrete controllable processing times, emphasizing the trade-offs between scheduling criteria and processing costs. Meanwhile, some studies have simultaneously considered aspects such as energy efficiency. For example, [Tsao et al. \(2020\)](#) developed an energy-efficient single-machine scheduling model with controllable job processing times under differential electricity pricing, applying fuzzy control-based optimization to reduce energy consumption. Multi-objective metaheuristic approaches have also been adopted for energy-efficient scheduling in distributed flow shops, integrating considerations of setup times and welding processes ([Wang et al., 2022](#)). Moreover, single-machine scheduling with controllable processing time have been employed with deteriorating jobs and convex resource functions, showing that jointly minimizing total weighted completion time and resource cost yields tractable structures and efficient algorithms ([Wang et al., 2024](#)).

Indeed, the scheduling under uncertainty and real-time control has been widely investigated in different industries. [Akturk and Ilhan \(2011\)](#) optimized single CNC machine scheduling with controllable processing times to minimize total weighted lateness, while [Kardos et al. \(2021\)](#) highlighted the importance of modeling downtime in dynamic environments to optimize throughput and reduce costs. [Li and Chang \(2022\)](#) advanced methods for controlling machine cycle times in multi-stage production, demonstrating improvements in cost and performance through real-time adjustments. In semiconductor manufacturing systems, [Liu et al. \(2022\)](#) developed dynamic scheduling approaches to improve responsiveness to urgent orders and uncertainties, while [Sun et al. \(2022\)](#) introduced neural network-based predictions of machining cycle times, enabling better scheduling in uncertain contexts.

Despite the scheduling with controllable processing times are implemented in general scheduling domains and some specific industries, the applications in semiconductor manufacturing have been rather limited. The characteristics and constraints of cluster tools, such as robot transfer synchronization or chamber sequences, make the application of more complex than in conventional environments. Integrating controllable processing times into under realistic characteristics remains challenging for semiconductor scheduling.

2.2. Semiconductor scheduling problem

For the application of semiconductor manufacturing, the scheduling of cluster tools is characterized by challenges such as wafer residency constraints, chamber cleaning, and concurrent processing of multiple wafer types. For example, [Li et al. \(2022\)](#) proposed efficient scheduling approaches for time-constrained single-arm cluster tools with condition-based chamber cleaning operations, balancing productivity and stringent cleaning requirements. [Wang et al. \(2023\)](#) developed an optimal cyclic scheduling framework for dual-arm cluster tools with wafer-residency constraints, demonstrating that configuring processing modules and robot waiting times can substantially improve throughput. [Yang et al. \(2025\)](#) formulated mixed integer programming models for single-arm cluster tools processing two wafer types concurrently, optimizing both release sequences and robot task sequences to enhance throughput and minimize wafer residency violations. [Li and Yang \(2025\)](#) proposed a cyclic scheduling framework for single-arm cluster tools handling multiple wafer types concurrently. The model schedules wafer release, robot operations, and chamber usage in fixed cycles and reduce residency violations. [Zhu et al. \(2025\)](#) implemented the application to dual-arm multi-cluster systems using an event-driven control strategy by dynamically adjusting robot handoffs and processing assignments in response to real-time wafer queue states to balances resource utilization across clusters.

Indeed, several studies have adopted RL approaches to address these scheduling challenges. [Kim and Lee \(2025\)](#) developed a DRL framework with adaptive search to optimize scheduling without relying on cyclic assumptions. In addition, some studies also extend the analysis of wafer delay dynamics in dual-arm cluster tools with periodic chamber cleaning, proposing strategies such as partial loading and timing control to mitigate delays and maintain high yield ([Yu & Lee, 2017](#); [Yu & Lee, 2020](#)). [Lee et al. \(2023\)](#) further optimized cleaning plans in dual-arm cluster tools, introducing the Dispersing and Gathering Cleaning (DGC) rule to minimize cycle times across various cleaning periods. [Zou](#)

[et al. \(2024\)](#) developed heuristic algorithms that significantly reduce wafer delay times caused by complex rotation and loading interactions in single-arm cluster tools. Indeed, DRL has also been widely adopted for other semiconductor scheduling and decision-making problems ([Chien & Lan, 2021](#)).

While these studies have significantly advanced cluster tool scheduling by addressing deterministic factors, the processing durations are assumed to be static values. Moreover, although RL methods have been applied to improve adaptability and decision-making under partially observable dynamics, their applications have primarily focused on optimizing sequencing and timing under fixed process time assumptions. There has been limited integration of controllable processing time decisions within cluster tool scheduling models with real-time uncertainty and operational constraints. Additionally, while prior research has significantly advanced cluster tool scheduling, most studies assume fixed processing times and emphasize objectives such as makespan minimization. To address these gaps, this study proposes a DRL framework that dynamically optimizes processing time configurations within realistic semiconductor production constraints.

3. Methodology

The proposed research framework is shown in [Fig. 1](#) and comprises two major components: the simulation model and the decision model. The simulation model is built using production resource data from each station, including machine configurations and process sequences, to replicate actual manufacturing scenarios. The decision model adopts RL to support decision-making and interacts closely with the simulation model. The RL agent is continuously trained on data generated from simulation, enabling iterative improvement. Through this integration, the framework aims to enhance production efficiency and minimize operational costs. The notation for the proposed model is illustrated in [Table 1](#).

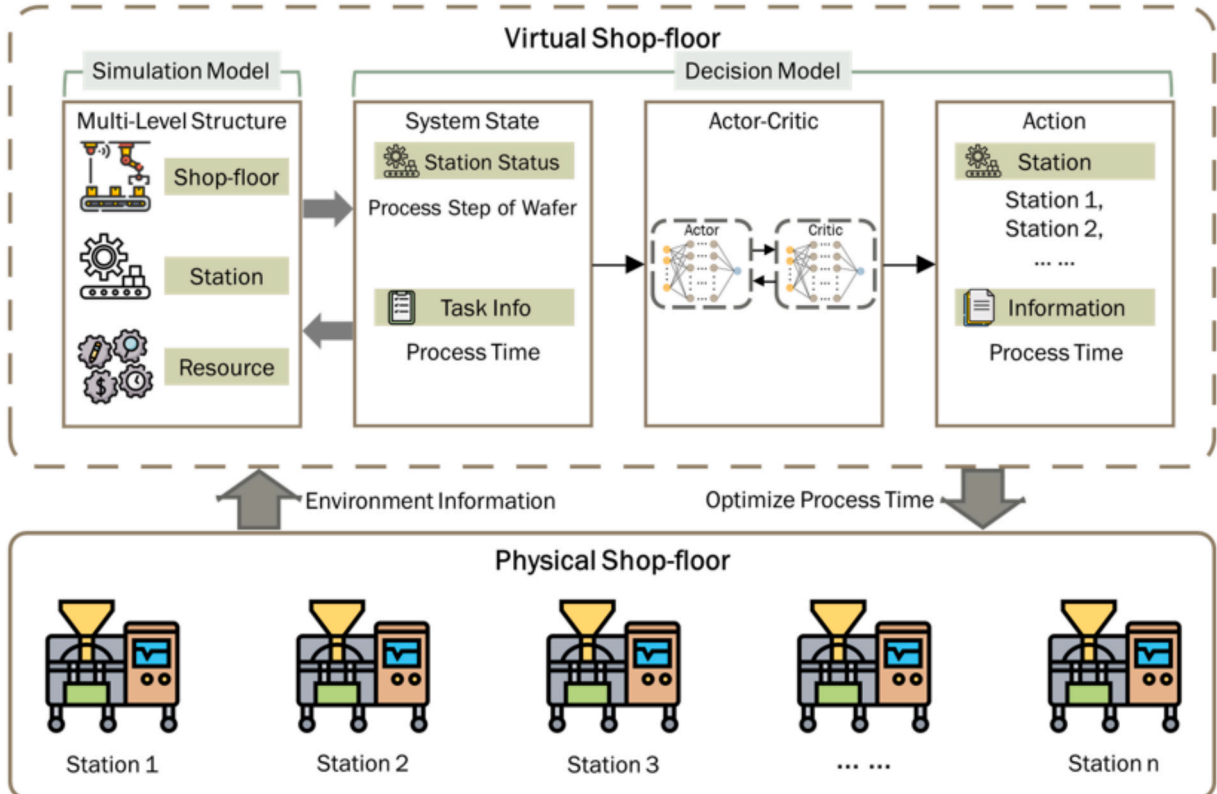


Fig. 1. Proposed research framework.

Table 1
Notation for the proposed model.

Set	
Symbol	Description
M	Set of processes for a tool, $M = \{M_1, M_2, \dots, M_k\}$
J	Set of wafers, $J = \{J_1, J_2, \dots, J_j\}$
B	Set of recipes, $B = \{B_1, B_2, \dots, B_b\}$
Q	Set of robots, $Q = \{q_1, q_2, \dots, q_v\}$
E_{M_k}	Subset of eligible machine for process M_k , $E_{M_k} = \{M_k^1, M_k^2, \dots, M_k^e\}$
O_{jh}	Operation set/sequence steps for wafer j with length h , $O_{jh} = \{o_{j0}, o_{j1}, \dots, o_{jh}\}$
q_{buffer}	Subset of processes served by buffer robots $q_{buffer} \subseteq M$
$q_{transfer}$	Subset of processes served by transfer robots $q_{transfer} \subseteq M$
Parameter	
Symbol	Description
n	Number of wafers to be processed
k	Number of processes
h	Number of operations in a wafer's route
b	Number of recipes
v	Number of robots
t	Decision time
δ	Lower-bound factor for processing time control ratio
φ	Processing time adjustment ratio
$target$	Target makespan specified by domain experts
u_k^b	Processing time of recipe B_b on machine M_k^e
Variable	
Symbol	Description
C_{max}	Makespan
C_{jh}	Completion time of wafer j at h -th operation
$u(t)$	Processing time vector at decision step t
$u_k^b(t)$	Controllable processing time at time step t

3.1. Problem definition

This study investigates the operation of PVD cluster tools. Fig. 2 outlines the key components and their functions within the cluster tool system. Each component is briefly described as follows:

1. Process Chamber: This unit performs core functions such as physical/chemical processing, inspection, pre-treatment or post-treatment, and may also serve as a temporary holding area for wafers. Multiple chambers with identical functions allow flexible wafer routing.

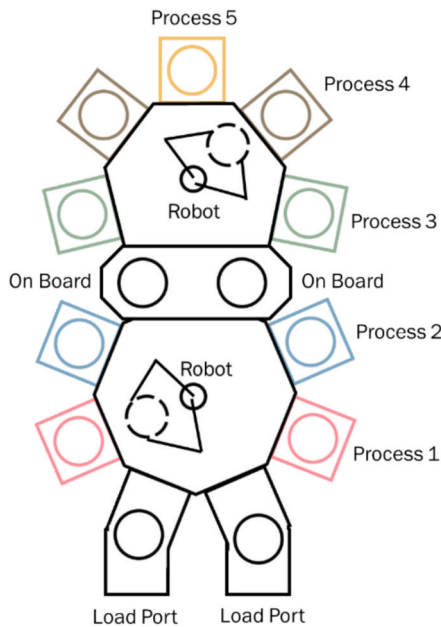


Fig. 2. An example of cluster tool with two mainframes.

2. On board Chamber: A chamber used solely for wafer transfer between modules. It does not conduct any wafer processing
3. Handler: A robotic arm responsible for wafer transport within the cluster tool. It moves wafers from the load port to process chambers and between chambers, then returns them after processing. Two transfer modes are used:
 - (a) Push Mode: After completing a process, the handler waits for the next chamber to finish before swapping wafers.
 - (b) Pull Mode: The handler moves a wafer to the next chamber as soon as that chamber becomes available.
4. Load Port: Also called the cassette module, which is the entry/exit point for wafers. Wafers are loaded by operators or AMHS, then isolated from the external environment to meet processing conditions. Once sealed, wafers are transferred into the system for processing

Indeed, the process of PVD cluster tools can be formulated as a hybrid flow shop scheduling problem. The definition of sets and notations are illustrated as follows. A tool consists of m processes, denote as $M = \{M_1, M_2, \dots, M_k, \dots, M_m\}$. There are n wafers to be processed, denoted as $J = \{J_1, J_2, \dots, J_j\}$. Each wafer J_j follows a predefined recipe, where $B_j \in B$, and $B = \{B_1, B_2, \dots, B_b\}$, in which b is the number of recipes. Each wafer J_j has a set of operations $O_{jh} = \{o_{j0}, o_{j1}, \dots, o_{jh}\}$, representing the sequence of steps required by its recipe. Based on the functionality of the process, the h -th operation of wafer J_j can be executed on a set of parallel machines $E_{M_k} = \{M_k^1, M_k^2, \dots, M_k^e\}$, where E_{M_k} represent the eligible machines for process M_k . The recipe B_b on machine M_k^e has a process time u_k^b , in which the value which depends on both the machine and the recipe. Two types of robots handle wafer movement, namely Transfer Robot and Buffer Robot. Each process M_k is assigned to one of these robot types, and the robot responsible for serving process M_k is defined as q_{buffer} or $q_{transfer}$.

The objective function can be formulated as Eq. (1), which is to minimize the squared deviation between the current makespan and the target value. In this context, the target makespan is a predefined value by the domain expert based on the needs of the manufacturing system. The value C_{max} is computed by Eq. (2), in which C_{jh} is the completion time of wafer J_j at its h -th operation.

$$\min f_1 = (C_{max} - target)^2 \quad (1)$$

$$C_{max} = \max_{j,h} C_{jh} \quad (2)$$

3.2. Simulation modeling

3.2.1. Data description

The data utilized for developing the DES model is presented in Fig. 3, which illustrates a hierarchical data structure composed of three levels: Process State, Wafer State, and Lot Processing State. The Process State stores all relevant information pertaining to individual processing steps. This includes details about the chambers that are available at each processing station, as well as the specific process parameters associated with each chamber, such as the processing time required for different operations. These data elements form the foundation for simulating realistic processing behavior in the cluster tool. The Wafer State captures the processing history of each individual wafer. It records all the processes that a wafer has undergone and maintains links to the corresponding Process State entries. In addition to process tracking, the Wafer State also stores identifying information, such as the wafer ID, enabling precise monitoring and traceability throughout the simulation.

At the highest level, the Lot Processing State contains comprehensive information about the entire lot undergoing production, which includes the status of all wafers within the lot, along with their respective Wafer States and associated process data. The Lot Processing State serves as a centralized structure that integrates the detailed histories and current

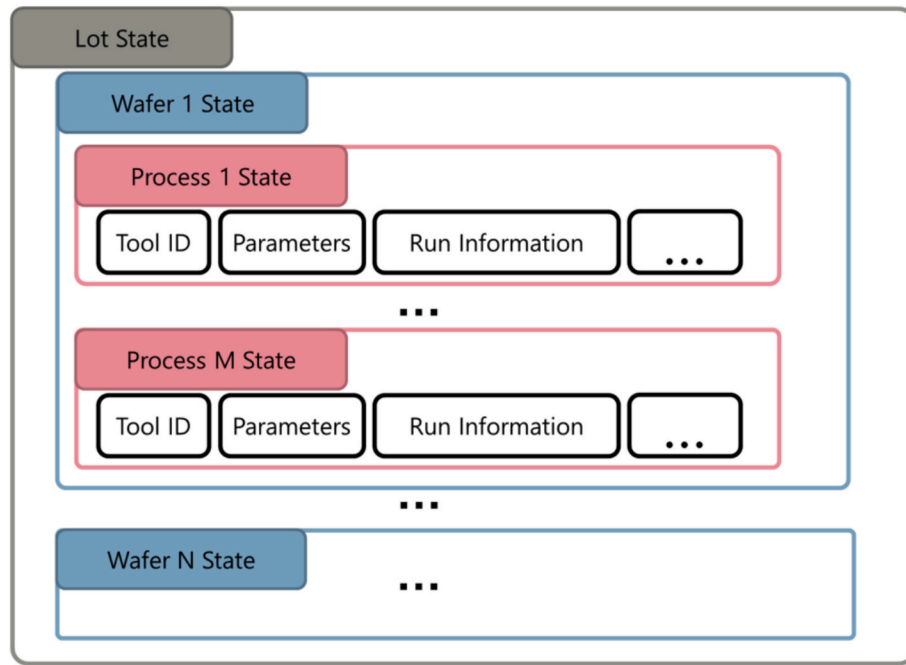


Fig. 3. The hierarchical structure of simulation input data.

statuses of all wafers, enabling the simulation to manage and evaluate lot-level behaviors, constraints, and performance metrics effectively. Through this hierarchical structure, the simulation model is able to reflect the complex interactions and dependencies among processes, wafers, and lots in a realistic manufacturing environment.

3.2.2. Simulation process flow

Based on the actual manufacturing process and the operational rules governing robot gripping and wafer transportation, a discrete event simulation model of the Cluster Tool with two mainframes is developed, as illustrated in Fig. 4. The load lock calls a robot q_v to grip the wafer and transport it to process M_k based on M_k served by buffer robots or transfer robots. Upon completion of the process, the system evaluates whether the current operation M_k represents the final stage in the sequence. If M_k is not the final process, the subsequent chamber M_{k+1} is examined to determine whether it resides within the same mainframe as M_k . If M_{k+1} belongs to a different mainframe, the system checks for the availability of at least one idle robot q_v before initiating wafer transfer. If no robot is available, the system re-evaluates this condition every 0.1 s until it is satisfied. Once a robot becomes available, it transfers the wafer to the on board station, which serves as an intermediate buffer between the two mainframes. Following the procedure, the system once again initiates a robot call, conditional on the availability of at least one idle chamber in M_{k+1} and one idle robot q_v . If these criteria are not met, the condition is repeatedly checked at 0.1 s intervals. Once fulfilled, the robot proceeds to grip and transfer the wafer to the target chamber. This sequence continues iteratively until the final process M_m is completed.

Indeed, the DES model is developed with the following assumptions to ensure tractability and focus on the core process logic. It is assumed that all chambers and operations are available and fully operational from the beginning, with no equipment failures occurring throughout the simulation. Each chamber is capable of processing only one wafer at a time, and once an operation begins, it must proceed without interruption until completion. The operations must follow a strict precedence order, meaning that a wafer's operation can only commence after all its preceding operations are completed. The model accounts for the time required for wafer transportation between processes, but does not consider quality control or defect-related interruptions, and thus assumes zero scrap generation. Additionally, processing times are assumed

to include any necessary setup time, and are operation-specific, with each process treated as independent of the others.

3.3. Deep reinforcement learning

Since the controllable processing time problem is modeled within a DES environment that represents robot transfers, waiting states, and chamber interactions in a PVD cluster tool, solving the proposed system by mathematical programming would require discretizing each process time and linearizing event-driven dependencies such as robot blocking and wafer transfer synchronization. These transformations may alter the temporal relationships among concurrent operations and substantially increase model complexity, leading to a combinatorial explosion in model size. Moreover, the asynchronous temporal blocking dynamics that arise from robot coordination and chamber availability could be a limitation while using these methods. To effectively manage the scheduling complexity of cluster tools and achieve a target makespan, this study introduces a DRL framework based on the Deep Deterministic Policy Gradient (DDPG) algorithm. The proposed process time control system (PTCS) agent interacts with the PVD simulation environment in a closed-loop manner. At each decision step t , the PTCS agent observes the current system state s_t , the agent selects a continuous-valued control action a_t based on the observation to adjust the process time decisions. Once the action is executed, the environment transitions to a new state s_{t+1} and provides an immediate reward r_t to reflect the effectiveness of the action in terms of reducing the deviation from the predefined target makespan. The reward signal guides the learning process and encourages the agent to discover time control policies that consistently lead to improved scheduling performance. Through repeated interactions, the PTCS agent learns to dynamically adjust process times in response to changing system conditions and operational constraints. Indeed, the simulation environment serves as the experimental platform, receiving the system state passed by the agent and executing actions based on the agent's choices, and the environment evaluates the adequacy of the agent's actions and provides feedback to the agent for learning.

3.3.1. State

The definition of states plays a critical role in shaping the agent's perception of the simulated production environment. In this study, the

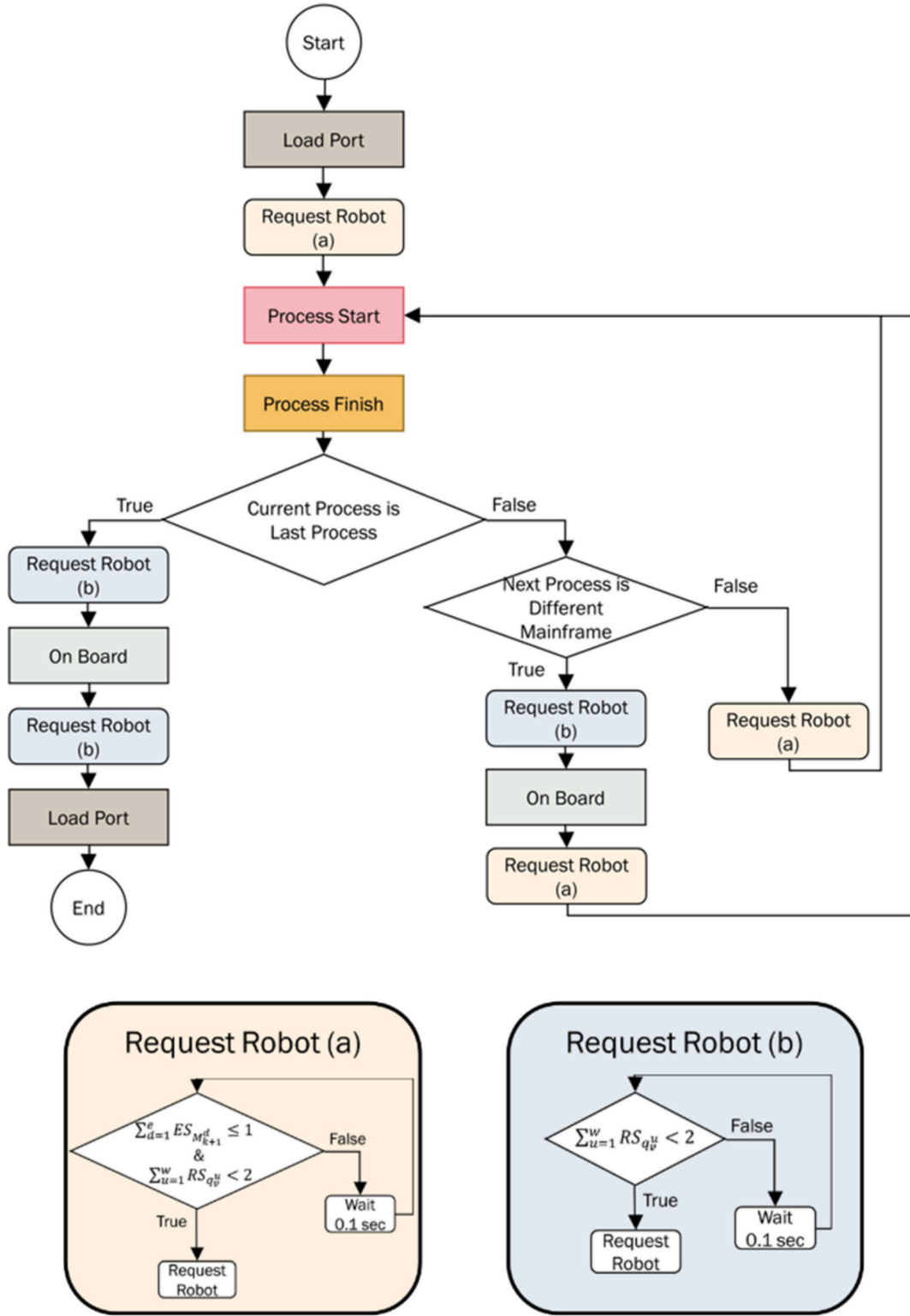


Fig. 4. Simulation process flow and robot handling.

agent relies on its learned policy to determine appropriate actions at each decision point, and this policy is continuously updated based on the historical sequence of states, selected actions, and the corresponding rewards. To ensure that the agent can effectively evaluate and improve its decisions, it is essential to include key system information in the state representation.

Accordingly, the states are designed to incorporate both the process

execution status and the performance feedback related to production objectives. Specifically, the state at time step $t+1$ defined as Eq. (3). Here, a_t denotes the action taken by the agent at time t , while the difference in makespan quantifies the gap between the previous and current production performance. The definition allows the agent to assess how the actions impact the overall makespan relative to the prior state, thereby enabling it to learn strategies that progressively reduce

makespan. Indeed, the makespan values are dynamically obtained through simulation. By integrating both action history and performance improvements into the state, the agent can capture temporal dependencies and develop policies that align with the objective of production control.

$$s_{t+1} = [a_t, \text{Makespan}_{t-1} - \text{Makespan}_t] \quad (3)$$

3.3.2. Action

The action space in this study is defined as the set of decisions specifying how to adjust the process time for each machine at the next time step. The agent determines the control inputs based on the current state, thereby adopting different strategies to improve system performance. At each time step t , the process time of machine M_k^e for recipe b can be adjusted proportionally by ratio φ . The adjustment ratio φ is selected from the interval $[-1, 1]$, allowing the process time to either increase or decrease within a bounded range. To maintain the feasibility of adjustments, a control ratio δ is introduced, which constrains the process time to remain within a specified deviation from its initial value. This value reflects the allowable temporal flexibility permitted by process engineers, typically determined based on equipment capabilities, film property tolerances, and control stability requirements. In this study, δ is a simulation parameter set as 0.15. The process time control vector at time t is defined as Eq. (4), where each element $u_k^b(t)$ represents the adjusted process time of machine M_k^e . The adjustment for each process time is computed by Eq. (5). The formulation ensures that the process time remains within the permissible range, preventing excessive deviations that could lead to infeasible production control. Therefore, the action a_t at time step t is defined as the process time control vector as shown in Eq. (6).

$$u(t) = [u_1^b(t), u_2^b(t), \dots, u_k^b(t)] \quad (4)$$

$$u_k^b(t) = \max\{u_k^b(1 - \delta), \min[u_k^b, u_k^b(t - 1) + \varphi]\} \quad (5)$$

$$a_t = u(t) = [u_1^b(t), u_2^b(t), \dots, u_k^b(t)] \quad (6)$$

3.3.3. Reward

The reward is defined as the change in makespan between consecutive decision steps. The immediate reward at time step t is expressed as Eq. (7), which allows the agent to directly measure the improvement in production efficiency resulting from the corresponding action. A positive reward indicates a reduction in makespan, while a negative reward reflects a deterioration in performance. The cumulative reward accumulated over the training episode is defined as the sum of all immediate rewards up to time t as shown in Eq. (8). By directly linking the reward signal to changes in makespan, the agent receives consistent feedback that reflects the primary performance criterion defined in the research objectives. Thus, the RL process is designed to guide the agent to find suitable strategies that reduce makespan within the training procedure.

$$r_t = \text{Makespan}_{t-1} - \text{Makespan}_t \quad (7)$$

$$R = r_1 + r_2 + \dots + r_t \quad (8)$$

3.3.4. Training procedure

A training procedure for process time control based on DDPG is adopted in this study (Lillicrap et al., 2015). The overall training framework is illustrated as follows. The input to the model consists of the original recipe information, while the output includes the weights of the actor and critic networks responsible for determining the control policy and value estimation, respectively. First, a replay memory Z is initialized to store transition tuples $(s_t, a_t, r_t, s_{t+1}, f_1)$. The critic network $Q(s, a|\theta^Q)$ and actor network $\mu(s|\theta^\mu)$ are initialized with random weights, and their corresponding target networks are created by copying the weights from the main networks. During each time step of an episode,

the actor network selects an action a_t from the available actions, which determines the adjustment of the machine's process time based on the current state s_t . This action specifies the adjustment of process times and is applied to the simulation environment. After executing the action, the environment transitions to a new state s_{t+1} , the immediate reward r_t , and makespan C_{max} are calculated. These results are used to compute the objective function f_1 , which measures the deviation from the target performance. The transition tuple is stored in replay memory for subsequent training. A batch of samples is periodically drawn from the memory to update the networks. The critic network is trained by minimizing the mean squared error between predicted and target Q-values. The actor network is updated using the policy gradient derived from the critic's evaluations. To improve learning stability, the weights of the target networks are softly updated by blending them with the main networks' weights using a factor τ . The training continues iteratively until the objective function f_1 close to the target value. The detailed pseudocode of DDPG algorithm is shown in Algorithm 1.

Algorithm 1 DDPG algorithm

Input: Original Recipe, target value target
Output: The weight of optimal actor parameter θ^Q , and critic θ^μ .
 Randomly initialize critic network $Q(s, a|\theta^Q)$ and actor $\mu(s|\theta^\mu)$ with weights θ^Q and θ^μ .
 Initialize target network Q' and μ' with weights $\theta^Q \leftarrow \theta^Q, \theta^{\mu'} \leftarrow \theta^\mu$
 Initialize replay buffer Z
For episode = 1, ..., M **do**
 Receive initial observation state s_1
for $t = 1, \dots, T$ **do**
 $a_t = \mu(s_t|\theta^\mu)$ select action according to the current policy
 Execute action a_t and observe reward r_t , observe new state s_{t+1} and makespan C_{max}
 Compute $f_1 = (C_{max} - \text{target})^2$.
 Store transition $(s_t, a_t, r_t, s_{t+1}, f_1)$ in Z
 Sample a random minibatch of N transitions $(s_i, a_i, r_i, s_{i+1}, f_1)$ from Z
 Set $y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1}|\theta^{\mu'}))|\theta^Q$
 Update critic by minimizing the loss: $L = \frac{1}{N} \sum_i (y_i - Q(s_i, a_i|\theta^Q))^2$
 Update the actor policy using the sampled policy gradient:
 $\nabla_{\theta^\mu} J \approx \frac{1}{N} \sum_i \nabla_a Q(s, a|\theta^Q)|_{s=s_i, a=\mu(s_i)} \nabla_{\theta^\mu} \mu(s|\theta^\mu)|_{s_i}$
 Update the target networks:
 $\theta^Q \leftarrow \tau \theta^Q + (1-\tau) \theta^Q, \theta^{\mu'} \leftarrow \tau \theta^{\mu'} + (1-\tau) \theta^{\mu'}$
if $f_1 == 0$ **then**
 break
end for
end for

4. Empirical study

4.1. Experimental setting

To validate the effectiveness of the proposed approach, simulation scenarios that are able to reflect actual production conditions are compared with performance against several established RL algorithms. The PVD equipment consists of 9 process chambers, 2 on boards, 3 load ports, 5 stations, and two sets of robotic arms are considered in this study. The corresponding process chambers for each workstation are depicted in Fig. 2.

Two experimental scenarios were designed to analyze and evaluate the proposed adjusted process time control approach. The cluster tool configuration used in the simulation was adapted from Dümmler (2004). The first scenario focused on a single-recipe experiment to isolate and assess the impact of process time adjustments on makespan and flow time. The second scenario incorporated multiple recipes executed sequentially to better reflect realistic production workflows. In the single-recipe scenario, only Recipe_A was utilized. By contrast, the multiple-recipe scenario included three recipes: Recipe_A, Recipe_B, and Recipe_C, processed in sequence. Each recipe consisted of five manufacturing steps, each corresponding to a specific workstation. To represent inherent variability in production, the process time at each step was modeled as a normal distribution. Additionally, each recipe involved the production of three lots, with each lot containing 25 wafers

to simulate typical batch manufacturing conditions. Following the simulation experiments, an empirical study was conducted using real-world data collected from a semiconductor foundry in Taiwan to further validate the effectiveness of the proposed method under practical operating conditions.

To benchmark the proposed method, the following RL algorithms, including the Asynchronous Advantage Actor Critic (A3C) (Mnih et al. 2016), the Normalized Advantage Function (NAF) (Gu et al. 2016), Twin-Delayed Deep Deterministic Policy Gradient (TD3) (Fujimoto et al. 2018), and the Soft Actor-Critic (SAC) (Haarnoja et al. 2018) were selected as the benchmarks of the proposed method. Four aspects are evaluated in the following experiments: (a) the convergence behavior and overall performance during training, (b) the proximity of the learned strategies to the target makespan within the solution space, (c) the flow time for each wafer from system entry to exit, and (d) the waiting time experienced by each wafer between processing steps.

Throughout the training process of the RL algorithm, the specific parameter configurations utilized can be found in Table 2. The training was implemented on a system equipped with an Intel(R) Core(TM) i7-4820 K CPU @ 3.70 GHz CPU 60 GB RAM, using Python 3.8 and Pytorch 1.12.

4.2. Scenario experiments

4.2.1. Single recipe results

For the single recipe scenario, each process time is modeled as a normally distributed random variable to reflect the inherent variability in processing times due to minor operational fluctuations. The standard deviation is set to 1, representing a small and consistent level of process uncertainty that is typical in stable production environments such as PVD.

With the specified parameters, we conducted training sessions for each algorithm and recorded the cumulative rewards at the end of every episode. The goal was to maximize cumulative rewards, which corresponds to achieving a process schedule as close as possible to the target makespan. The results as shown in Fig. 5 demonstrate the average cumulative reward over ten independent runs for each method, which indicates that DDPG and NAF achieve higher cumulative rewards compared to A3C, TD3, and SAC. This suggests that deterministic policy gradient methods can be effective in learning efficient control strategies for the proposed problem. However, the DDPG model exhibits considerable fluctuations during training, indicating instability in policy convergence. Indeed, this behavior can be attributed to the deterministic nature of its policy updates combined with continuous action space exploration. In the context of process time adjustment, such variability is expected. The algorithm samples diverse action combinations to identify optimal process time configurations. This results in reward oscillations as the policy iteratively refines toward higher-performing solutions. Therefore, while the training trajectory may appear less stable compared to algorithms with more conservative exploration strategies, this

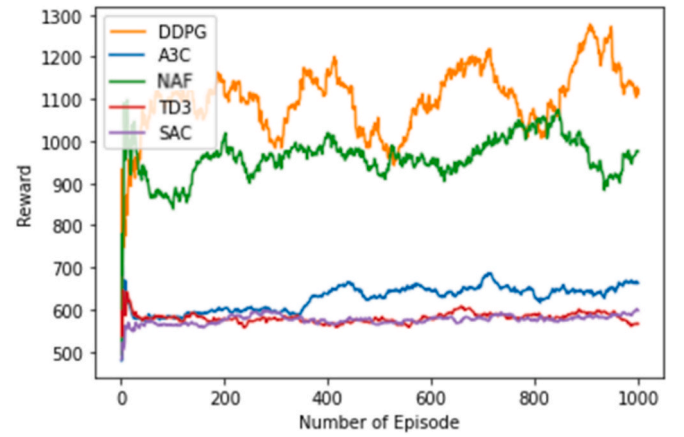


Fig. 5. Cumulative reward for single recipe scenario.

characteristic enables DDPG to more effectively search the solution space and converge to configurations that are closer to the target performance. Moreover, A3C, TD3, and SAC demonstrate a stable learning behavior but converge to lower reward levels, which indicates that these algorithms may be less suited for capturing the specific process dynamics under the current problem formulation.

The deviations from the target makespan for each algorithm are shown in Fig. 6. It can be observed that the median of the DDPG solutions is located close to zero, indicating that most solutions are near the target value. The interquartile range (IQR) of the DDPG solutions is also smaller compared to the other methods, suggesting that the solutions found by DDPG are more concentrated and consistent. In contrast, the medians of the other four methods are further from zero, and the IQRs are larger, particularly for TD3 and SAC. Indeed, the range of solutions obtained by these methods is relatively distributed. The results demonstrate that despite exhibiting higher variability during training, the DDPG approach is capable of converging to high-quality solutions with less dispersion across independent runs.

4.2.2. Multiple recipe results

In the multiple-recipe scenario, where production involved repeating a single recipe in sequence, this scenario required sequential manufacturing of three distinct recipes. After completing the production of all lots for Recipe_A, the system transitioned to producing Recipe_B, followed by Recipe_C. Throughout the simulation, the machines and robots were allocated according to the predefined scheduling rules to process wafers in the correct sequence. One key difference observed in this scenario was the presence of gaps during transitions between

Table 2
Parameter settings of the DDPG algorithm.

Parameter	Value
Number of run time	10
Number of episodes	1000
Max time steps of each episode	1000
Minibatch size	64
Replay memory size	5×10^6
Number of state space	16
Number of action space	15
Processing time control ratio	0.15
The learning rate of actor	0.001
The learning rate of critic	0.001
Target network update rate	10^{-3}
Reward discount factor	0.99
Simulation times	30,000(s)

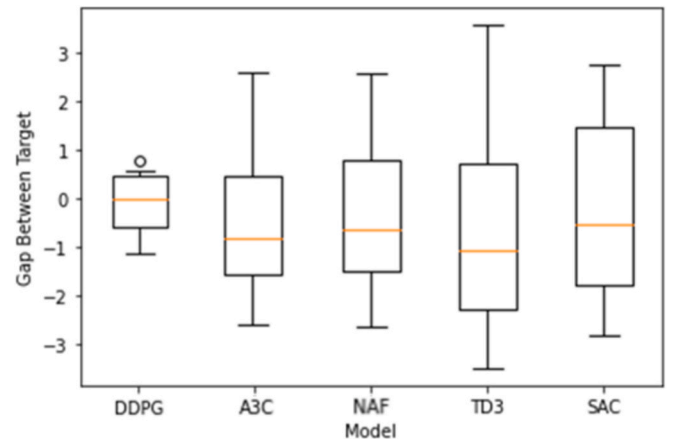


Fig. 6. Distribution comparison for single recipe scenario.

recipes, which indicates recipe sequencing and workstation configuration can introduce additional complexity compared to the single-recipe setting. Managing the gaps and synchronization between recipes becomes essential.

The training process for each algorithm used the specified parameters and recorded the cumulative rewards at the end of every training episode. The RL agent was trained to maximize cumulative rewards, where higher rewards indicate solutions that more closely approach the target makespan. After 1,000 training episodes, the results are presented in Fig. 7. Results show that the DDPG achieves similar performance to the single-recipe scenario in terms of cumulative rewards. However, as the solution space expands in the multiple-recipe setting, DDPG demonstrates a clear advantage over the other methods. With more process times adjustable across different recipes, the solution space becomes significantly larger, providing greater flexibility to explore alternative configurations. This expanded space enables the DDPG algorithm to identify higher-quality solutions and contributes to a cumulative reward curve that is both higher and more stable over the course of training.

The solutions of each algorithm were further ranked based on the magnitude of their deviation from the target, as illustrated in Fig. 8. With a greater number of adjustable process time parameters available in the multiple-recipe scenario, the search space is substantially expanded, providing increased opportunities to discover more efficient process time combinations. Indeed, the results demonstrate that the DDPG model produces solutions with a median deviation close to zero, which indicates that the makespan is aligned with the target value. Furthermore, the IQR for the DDPG solutions also suggests a high level of consistency and concentration among the solutions.

Based on the original process time configurations and the solutions identified by DDPG, simulations were conducted to evaluate production performance. Fig. 9(a) shows the flow time for each wafer from entry into the system until exit, while Fig. 9(b) illustrates the waiting time for each wafer after each process step until it leaves the machine. In these figures, solid lines represent the results under the original process time settings, and dashed lines correspond to the results obtained from the optimized process times, with different colors indicating different recipes. The solutions generated by the DDPG approach lead to an overall reduction in both flow time and waiting time across all recipes. In particular, recipes B and C exhibit more substantial improvements in throughput and process efficiency. Furthermore, the results show that improvements in process performance are not necessarily proportional to the scale of process time adjustments. In some cases, even minor modifications in process times can yield significant reductions in waiting time and makespan, which also indicates the importance of systematic optimization rather than relying solely on intuitive adjustments or uniform reductions in processing duration.

A sensitivity analysis, as shown in Fig. 10, is further conducted to

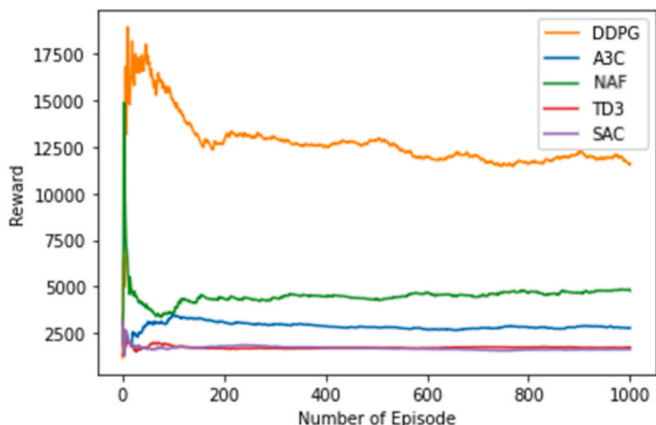


Fig. 7. Cumulative reward for multiple recipe scenario.

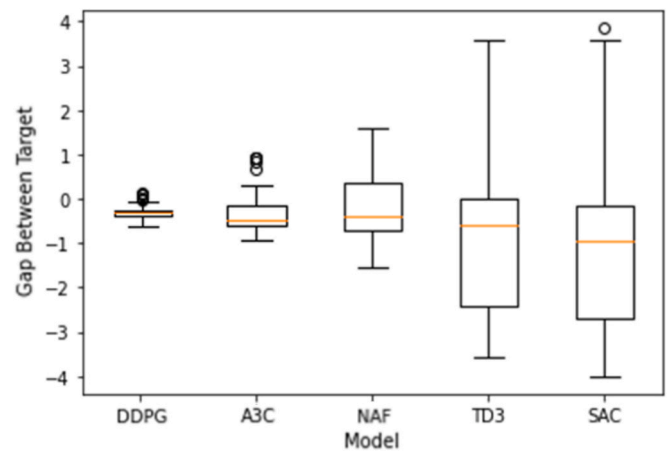


Fig. 8. Distribution comparison for multiple recipe scenario.

validate the result under different target makespan values. The targets are determined based on reducing the makespan obtained under the original process time configuration by a specific percentage, denoted as the δ ratio. This ratio defines the level of improvement required relative to the baseline makespan and the degree of adjustment needed for each process time. Three levels were analyzed in which the δ ratio was set to 0.10, 0.15, and 0.20, respectively. Fig. 5.12 illustrates the results for flow time and waiting time under the three target levels. Under the same simulation framework, the results show that when the δ ratio was set to 0.10 and 0.15, the overall flow time and waiting time remained relatively similar, indicating that moderate target reductions can be achieved without introducing significant variability in system performance. In contrast, when the δ ratio was increased to 0.20, the results represent a more substantial reduction in makespan. The improvement in flow time and waiting time became more pronounced. However, this also reflects a narrower feasible adjustment range, which limits the exploration space and increases the risk of process constraints impacting machine operations. The results also show the trade-off between pursuing more aggressive performance targets and maintaining operational stability. Indeed, an appropriate target makespan reduction is critical for achieving balanced improvements without compromising the robustness of the scheduling process.

4.3. Empirical experiment

For empirical experiments, detailed production data were collected from a real-world semiconductor foundry during the period from June 14 to June 16, 2023. Consecutive wafer records were selected to form complete lots for each recipe, and the data were compiled to calculate the mean and standard deviation of processing times for each step of each recipe, as illustrated in Table 3. Each recipe was scheduled to produce three lots of wafers, with 25 wafers per lot. The DDPG-Based PTCS developed in this study was then applied to identify optimized process time configurations capable of achieving this reduction target. Based on the actual collected process data, the original process time of the recipes was used as a baseline for comparison. The simulation results comparing the optimized solutions and the original process time settings are presented in Fig. 11(a), showing the flow time of each wafer from entry to exit.

The results demonstrate that the optimized solutions identified by the DDPG consistently reduced total process time across all recipes compared to the original configurations. Furthermore, the waiting times between process steps were also substantially decreased, as shown in Fig. 11(b). The results highlight that even in real production scenarios with inherent variability, systematic adjustment of process times through reinforcement learning can effectively enhance the makespan reduction and improve overall production efficiency.

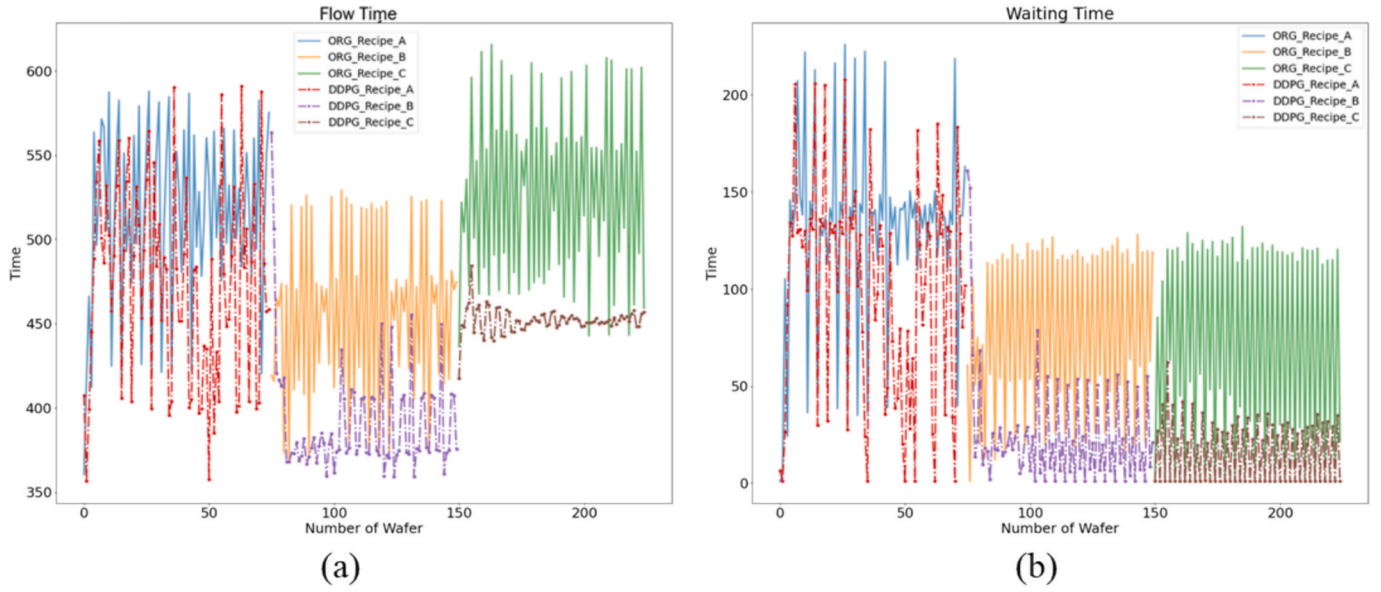


Fig. 9. Comparison of (a) flow time and (b) waiting time for multiple recipe experiments.

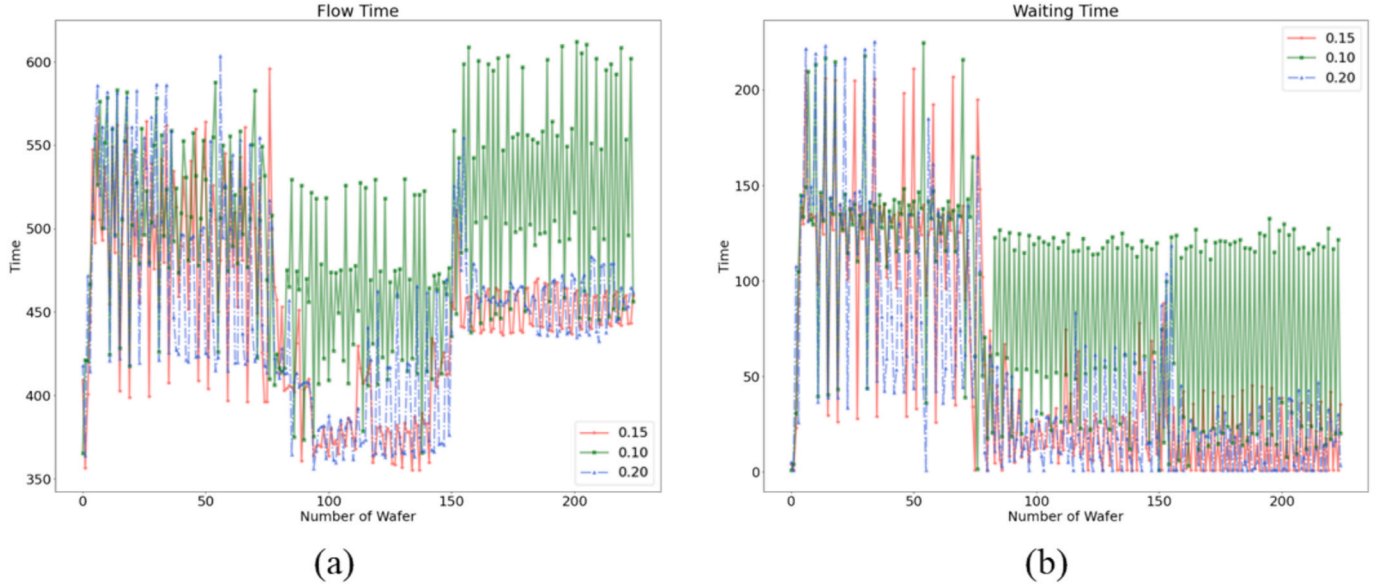


Fig. 10. Results of sensitivity analysis (a) flow time and (b) waiting time.

Table 3

Process time of each recipe for empirical experiment.

Process Time (sec)	Process 1 Mean (Std)	Process 2 Mean (Std)	Process 3 Mean (Std)	Process 4 Mean (Std)	Process 5 Mean (Std)
Recipe_D	86 (0.92)	105 (0.37)	116 (0.33)	62 (0.37)	30 (1.04)
Recipe_E	85 (0.92)	105 (0.37)	116 (0.33)	116 (0.30)	30 (1.04)
Recipe_F	85 (0.92)	105 (0.37)	58 (0.47)	116 (0.30)	30 (1.04)

4.4. Discussion

The results obtained across the single-recipe, multiple-recipe, and empirical experiments highlight the advantages of applying RL for process time adjustment in semiconductor manufacturing. The results summarized in Table 4 show the improvements compared to the original

configuration. In all cases, the method demonstrated clear improvements in both wafer cycle time and waiting time to show the adaptability to different production conditions. To verify the observed improvements, a one-sample *t*-test was performed for each performance indicator. Each DDPG result was compared against the original (ORG) benchmark to evaluate whether the learned policy achieved significant reductions. The results show that all differences are statistically significant across makespan, flow time, and waiting time. These outcomes indicate that the DDPG-based controller consistently outperforms the baseline configuration, and that the observed improvements are not due to random variation but reflect a stable and reproducible learning effect.

It is worth noting that the relative improvement in waiting time was more substantial in scenarios with greater variability among processes and recipes. The finding suggests that RL is particularly effective in environments where traditional static scheduling struggles to efficiently coordinate resources and minimize idle periods. The significant gains observed in the empirical scenario further support the practical

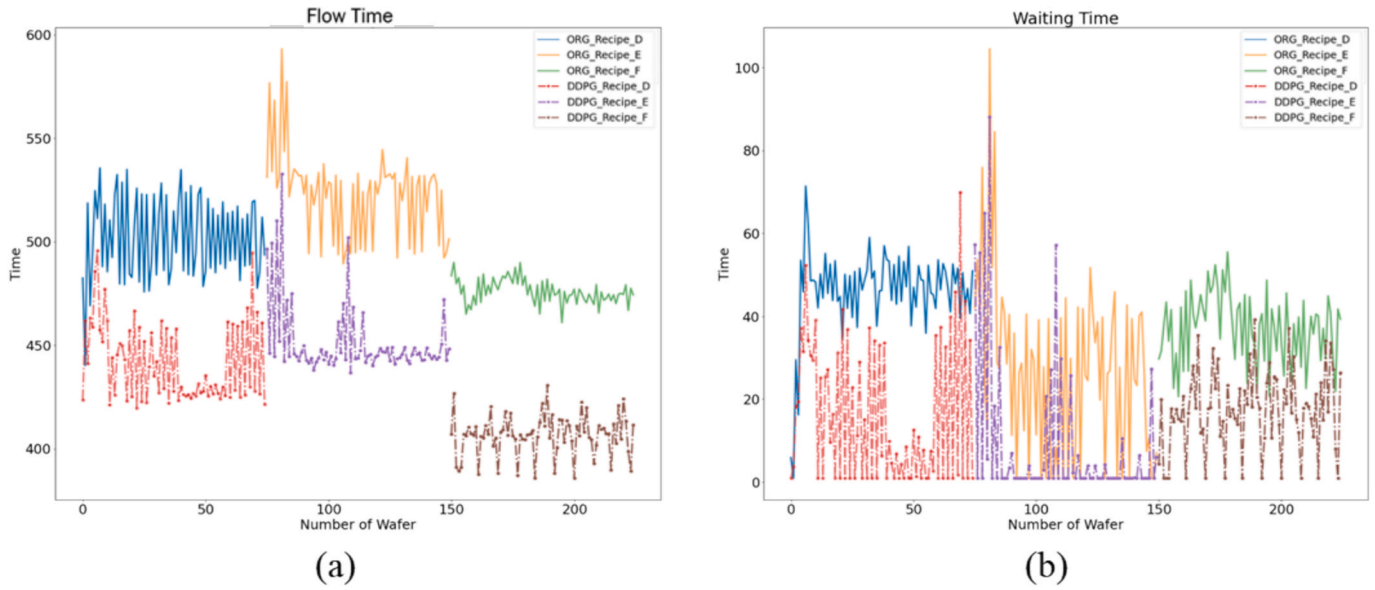


Fig. 11. Comparison of (a) flow time and (b) waiting time for empirical experiments.

Table 4
Improvement of the proposed method.

		Results			One sample <i>t</i> -test		
		Makespan	Summary of Flow Time	Summary of Waiting time	Makespan	Flow Time	Waiting time
Single Recipe	Original	6,099.74	38,445.95	9,651.28	$p < 0.001$	$p < 0.001$	$p < 0.001$
	DDPG	5,488.88 (43.92)	34,732.29 (347.32)	8,734.47 (104.81)			
	Improvement (%)	10.00 %	9.66 %	9.50 %			
Multiple Recipe	Original	16,582.29	111,215.16	19,966.64	$p < 0.001$	$p < 0.001$	$p < 0.001$
	DDPG	14,924.23 (149.24)	99,779.11 (1097.57)	12,299.44 (184.49)			
	Improvement (%)	10.00 %	10.28 %	38.40 %			
Empirical Experiment	Original	15,800.00	112,574.53	5,237.37	$p < 0.001$	$p < 0.001$	$p < 0.001$
	DDPG	14,220.21 (142.2)	97,426.67 (1169.12)	3,585.75 (64.54)			
	Improvement (%)	10.00 %	13.46 %	31.54 %			

relevance of the approach, which shows that even under realistic process fluctuations, reinforcement learning can deliver measurable performance benefits. By leveraging the proposed framework rather than relying solely on expert heuristics, domain users can systematically uncover more efficient scheduling configurations. Furthermore, the approach provides a flexible framework that can be further adapted to different objectives such as lowering wafer waiting times or improving throughput stability.

From a managerial perspective, the results also indicate several practical implications. First, the RL-based processing time control mechanism can be implemented as a decision-support module within existing manufacturing execution systems (MES), enabling engineers to continuously fine-tune process durations in response to real-time workload variations without modifying the underlying dispatching rules. Second, the framework allows fab managers to conduct the analyses through simulation to evaluate how different process-time control ratios or learning policies affect cycle time and tool utilization before deployment. Finally, the learning mechanism supports continuous improvement. After system deployment, the model can progressively accumulate operational experience and adapt to changes in recipes, equipment conditions, and throughput objectives, thereby reducing

dependence on manual parameter tuning.

5. Conclusion

This study developed a RL framework integrated with simulation to optimize process times and improve production performance in semiconductor manufacturing. By constructing a virtual environment that reflects the production conditions of PVD cluster tools and employing a DDPG-based approach, the proposed method enables efficient exploration of high-dimensional continuous action spaces and delivers reliable recommendations for process adjustments. Experimental results demonstrated that the proposed DDPG-based PCTS outperformed other RL methods in reducing wafer flow time and waiting time across scenarios. Sensitivity analysis further revealed that the target reduction ratios significantly influence solution quality and system stability. Indeed, the proposed framework not only demonstrates technical effectiveness in optimizing controllable process times, the proposed framework provides a practical foundation for integrating adaptive decision-making for real-world deployment in semiconductor fabs.

Nevertheless, some limitations are acknowledged. Although the empirical validation verifies the feasibility in a realistic production

setting, the current scope remains restricted to a single foundry and a limited observation period. Future research can extend the validation period and include additional empirical datasets to examine scalability and industrial applicability further. Moreover, future work can be done by continuously monitoring and updating to reflect changes in the practical environment based on the digital twin concept to make proactive adjustments in response to process variability.

CRedit authorship contribution statement

Tzu-Yen Hong: Writing – review & editing, Writing – original draft, Visualization, Validation, Methodology, Investigation, Formal analysis, Conceptualization. **Chia-Yu Hsu:** Writing – review & editing, Supervision, Resources, Project administration, Methodology, Funding acquisition. **Chia-Ling Chen:** Validation, Methodology, Investigation, Formal analysis.

Acknowledgement

This research was supported by the National Science and Technology Council, Taiwan (NSTC 112-2622-E-011-021; NSTC 113-2221-E-027-111).

Data availability

The data that has been used is confidential.

References

- Akturk, M. S., & Ilhan, T. (2011). Single CNC machine scheduling with controllable processing times to minimize total weighted tardiness. *Computers & Operations Research*, 38(4), 771–781.
- Atan, M. O., & Akturk, M. S. (2008). Single CNC machine scheduling with controllable processing times and multiple due dates. *International Journal of Production Research*, 46(21), 6087–6111.
- Chen, W. C., & Chien, C. F. (2011). Evaluating capacity pooling strategy in semiconductor manufacturing: A productivity perspective study. *International Journal of Production Research*, 49(12), 3635–3652.
- Cheng, Q., Kan, H., Ren, J., Liu, Z., Zhang, Y., & Cheng, C. (2024). Multi-objective optimization method for cross-workshop linkage production of partially flexible free-forging with forward single-machine scheduling. *Computers & Industrial Engineering*, 196, Article 110508.
- Chien, C. F., & Chen, C. H. (2007). A novel timetabling algorithm for a furnace process for semiconductor fabrication with constrained waiting and frequency-based setups. *OR Spectrum*, 29, 391–419.
- Chien, C.-F., Kuo, C.-J., & Yu, C.-M. (2020). Tool allocation to smooth work-in-process for cycle time reduction and an empirical study. *Annals of Operations Research*, 290(1), 1009–1033.
- Chien, C. F., & Lan, Y. B. (2021). Agent-based approach integrating deep reinforcement learning and hybrid genetic algorithm for dynamic scheduling for Industry 3.5 smart production. *Computers & Industrial Engineering*, 162, Article 107782.
- Dümmler, M. (2004). *Modeling and optimization of cluster tools in semiconductor manufacturing*. Lehrstuhl für Verteilte Systeme: Institut für Informatik.
- Fujimoto, S., Hoof, H., & Meger, D. (2018). Addressing Function Approximation Error in Actor-Critic Methods. *International Conference on Machine Learning*, 1587–1596.
- Gu, S., Lillicrap, T., Sutskever, I., & Levine, S. (2016). Continuous Deep Q-Learning with Model-based Acceleration. *International Conference on Machine Learning*, 2829–2838.
- Haarnoja, T., Zhou, A., Hartikainen, K., Tucker, G., Ha, S., Tan, J., Kumar, V., Zhu, H., Gupta, A., and Abbeel, P. (2018), "Soft Actor-Critic Algorithms and Applications," *arXiv preprint arXiv:1812.05905*.
- Hong, T. Y., & Chien, C. F. (2020). A simulation-based dynamic scheduling and dispatching system with multi-criteria performance evaluation for Industry 3.5 and an empirical study for sustainable TFT-LCD array manufacturing. *International Journal of Production Research*, 58(24), 7531–7547.
- Hsieh, P. H., Yang, S. J., & Yang, D. L. (2015). Decision support for unrelated parallel machine scheduling with discrete controllable processing times. *Applied Soft Computing*, 30, 475–483.
- Hu, W., Liu, M., Dong, M., Liu, T., Zhang, Y., & Cheng, G. (2023). A greedy-based crow search algorithm for semiconductor final testing scheduling problem. *Computers & Industrial Engineering*, 183, Article 109423.
- Kardos, C., Lafiamme, C., Gallina, V., & Sihni, W. (2021). Dynamic scheduling in a job-shop production system with reinforcement learning. *Procedia CIRP*, 97, 104–109.
- Kim, H. J., & Lee, J. H. (2025). Scheduling Cluster Tools for concurrent Processing: Deep Reinforcement Learning with Adaptive Search. *IEEE Transactions on Automation Science and Engineering*, 22, 3783–3796.
- Lee, T. G., Yu, T. S., & Lee, T. E. (2023). Cleaning plan optimization for dual-armed cluster tools with general chamber cleaning periods. *IEEE Transactions on Automation Science and Engineering*, 20(3), 1890–1906.
- Li, C., & Chang, Q. (2022). Hybrid feedback and reinforcement learning-based control of machine cycle time for a multi-stage production system. *Journal of Manufacturing Systems*, 65, 351–361.
- Li, C., Yang, F., & Zhen, L. (2022). Efficient scheduling approaches to time-constrained single-armed cluster tools with condition-based chamber cleaning operations. *International Journal of Production Research*, 60(11), 3555–3568.
- Li, X., & Yang, W. (2025). Cyclic scheduling of multi-type wafers concurrent processing in single-arm cluster tools with residency time constraints. *Expert Systems with Applications*, 269, Article 126443.
- Lillicrap, T. P., Hunt, J. J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., Silver, D., & Wierstra, D. (2015). Continuous control with deep reinforcement learning. *arXiv preprint arXiv:1509.02971*.
- Liu, J., Qiao, F., Zou, M., Zinn, J., Ma, Y., & Vogel-Heuser, B. (2022). Dynamic scheduling for semiconductor manufacturing systems with uncertainties using convolutional neural networks and reinforcement learning. *Complex & Intelligent Systems*, 8(6), 4641–4662.
- Mnih, V., Badia, A. P., Mirza, M., Graves, A., Lillicrap, T., Harley, T., Silver, D., & Kavukcuoglu, K. (2016). Asynchronous Methods for Deep Reinforcement Learning. *International Conference on Machine Learning*, 1928–1937.
- Shabtay, D. (2022). Single-machine scheduling with machine unavailability periods and resource dependent processing times. *European Journal of Operational Research*, 296(2), 423–439.
- Shabtay, D., & Steiner, G. (2007). A survey of scheduling with controllable processing times. *Discrete Applied Mathematics*, 155(13), 1643–1666.
- Shabtay, D., & Zof, M. (2018). Single machine scheduling with controllable processing times and an unavailability period to minimize the makespan. *International Journal of Production Economics*, 198, 191–200.
- Sun, C., Dominguez-Caballero, J., Ward, R., Ayvar-Soberanis, S., & Curtis, D. (2022). Machining cycle time prediction: Data-driven modelling of machine tool feedrate behavior with neural networks. *Robotics and Computer-Integrated Manufacturing*, 75, Article 102293.
- Tsao, Y. C., Thanh, V. V., & Hwang, F. J. (2020). Energy-efficient single-machine scheduling problem with controllable job processing times under differential electricity pricing. *Resources, Conservation and Recycling*, 161, Article 104902.
- Wang, G., Li, X., Gao, L., & Li, P. (2022). An effective multi-objective whale swarm algorithm for energy-efficient scheduling of distributed welding flow shop. *Annals of Operations Research*, 1–33.
- Wang, J., Liu, C., Zhou, M., Leng, T., & Albeshri, A. (2023). Optimal cyclic scheduling of wafer-residency-time-constrained dual-arm cluster tools by configuring processing modules and robot waiting time. *IEEE Transactions on Semiconductor Manufacturing*, 36(2), 251–259.
- Wang, J. B., Wang, Y. C., Wan, C., Lv, D. Y., & Zhang, L. (2024). Controllable processing time scheduling with total weighted completion time objective and deteriorating jobs. *Asia-Pacific Journal of Operational Research*, 41(03), Article 2350026.
- Yang, D. L., Lai, C. J., & Yang, S. J. (2014). Scheduling problems with multiple due windows assignment and controllable processing times on a single machine. *International Journal of Production Economics*, 150, 96–103.
- Yang, F., Li, C., Zhen, L., Zhang, C., & Wu, N. (2025). Scheduling residency time-constrained single-armed cluster tools with two-wafer types via mixed integer programming model. *Expert Systems with Applications*, 128209.
- Yu, T. S., & Lee, T. E. (2017). Scheduling dual-armed cluster tools with chamber cleaning operations. *IEEE Transactions on Automation Science and Engineering*, 16(1), 218–228.
- Yu, T. S., & Lee, T. E. (2020). Wafer delay analysis and control of dual-armed cluster tools with chamber cleaning operations. *International Journal of Production Research*, 58(2), 434–447.
- Zhang, L., Lin, Y., Xu, C., & Liu, M. (2024). A new EDA algorithm combined with Q-learning for semiconductor final testing scheduling problem. *Computers & Industrial Engineering*, 193, Article 110259.
- Zou, C., Zhang, S., Zeng, S., Gu, L., & Li, J. (2024). Wafer Delay Minimization in Scheduling Single-arm Cluster Tools with Two-Space Process Modules. *Mathematics*, 12(12), 1783.
- Zhu, Q., Zhang, G., Hou, Y., & Wu, N. (2025). Scheduling a dual-arm robotic two-cluster tool in the event of a process module failure. *Expert Systems with Applications*, 129415.